SHRINCS specification

Blockstream Research

February 3, 2026

Abstract

This document specifies **SHRINCS** (Stateful Hash-based Robust Integrated NIST-Compatible Signatures), a post-quantum signature scheme optimized for Bitcoin. SHRINCS combines efficient stateful hash-based signatures (unbalanced XMSS with WOTS+C) for normal operation with a stateless fallback (SPHINCS+ variant) for scenarios when state management failed. The scheme achieves minimal 324-byte signatures during normal stateful operation while maintaining the robustness of stateless signatures as a backup when state is corrupted or lost.

Contents

1 Motivation **3**
1.1 SHRINCS . . . 3
1.2 Limitations and Practical Considerations . . . 3

2 Notation **4**

3 Overview **5**
3.1 Key Pair Derivation and Size . . . 6
3.2 Signature Size and Verification Cost . . . 6
3.2.1 Stateful . . . 6
3.2.2 Stateless . . . 6

4 Parameters **7**

5 Underlying Functions and Addresses **7**
5.1 Hash Function . . . 7
5.2 Tweakable Hash Function . . . 7
5.3 Pseudorandom Functions . . . 7
5.4 Message Hash Function . . . 8
5.5 Tweak Structure and Domain Separation . . . 8
5.5.1 Address Types . . . 8
5.5.2 Address Manipulation Functions . . . 9
5.5.3 Type-Dependent Words . . . 9

6 WOTS+C **9**
6.1 The Target Condition . . . 10
6.2 Base-w Representation . . . 10
6.3 Chain Function . . . 11
6.4 Key Generation . . . 12
6.5 Messsage Digest Grinding . . . 12
6.6 Signature Generation . . . 13
6.7 Signature Verification (Public Key Recovery) . . . 14
6.8 Contribution to the Signature Size and Verification Complexity . . . 15
6.9 Implementation notes . . . 15

7 XMSS **15**
7.1 XMSS Tree Structure . . . 15
7.2 XMSS TreeHash . . . 15
7.3 XMSS Root Computation . . . 16
7.4 XMSS Authentication Path Generation . . . 16
7.5 XMSS Public Key Recovery from Signature . . . 17
7.6 XMSS Signature Generation (Stateless Layer) . . . 17

8 UXMSS. The Stateful Tree **18**
8.1 UXMSS Tree Structure . . . 18
8.2 UXMSS TreeHash . . . 18
8.3 UXMSS Root Computation . . . 19
8.4 UXMSS Authentication Path Generation . . . 19
8.5 UXMSS Public Key Recovery from Signature . . . 20
8.6 UXMSS Signature Generation . . . 21

9 FORS+C **21**
9.1 FORS Structure . . . 21
9.2 Grinding Condition . . . 22
9.3 Message to Indices . . . 22
9.4 Grinding . . . 22
9.5 Secret Key Generation . . . 23
9.6 TreeHash . . . 23
9.7 FORS Authentication path . . . 24
9.8 Signature generation . . . 24
9.9 Public Key Recovery . . . 25

10 SHRINCS **26**
10.1 Key Generation . . . 26
10.2 Recovery from Seed . . . 27
10.3 Statefull signing . . . 27
10.4 Stateless signing . . . 28
10.5 Index Parsing . . . 28
10.6 Statefull verification . . . 29
10.7 Stateless verification . . . 30
10.8 Unified Verification . . . 31

11 Data structures **31**
11.1 Secret Key Format . . . 31
11.2 Public Key Format . . . 31
11.3 Stateful Signature Format . . . 31
11.4 Stateless Signature Format . . . 31

1 Motivation

Current Bitcoin signature schemes (ECDSAECDSA[**ecdsa**] and Schnorr[**bip341**] over secp256k1) are vulnerable to quantum attacks via Shor's algorithm[**shor**]. Hash-based signature schemes offer post-quantum security relying only on the security of cryptographic hash functions, which are believed to resist quantum attacks apart from Grover's quadratic speedup (mitigated by doubling hash output lengths).

The key challenges for deploying hash-based signatures in Bitcoin are:

1. Signature size: NIST-standardized SLH-DSA signatures [**slh-dsa**] range from 7KB to 50KB, significantly larger than current 64-byte Schnorr signatures. This increases transaction fees and reduces the network's throughput.

2. Stateful vs. Stateless tradeoff: Stateful schemes (e.g., XMSS [**xmss**]) offer compact signatures but require careful state management to prevent key reuse. Additionally, Bitcoin users expect to restore wallets from static seed phrases (BIP-39[**bip-39**]). Pure stateful schemes break this model since signing state cannot be recovered from a seed alone, the signer must know which one-time keys have already been used. Stateless schemes (e.g., SLH-DSA) are robust but produce large signatures.

1.1 SHRINCS

SHRINCS (Stateful Hash-based Robust Integrated NIST-Compatible Signatures) is a hybrid post-quantum signature scheme that combines the efficiency of stateful signatures with the robustness of stateless ones:

- **Efficient stateful path.** During normal operation with intact state, SHRINCS uses an Unbalanced XMSS (UXMSS) tree with WOTS+C one-time signatures. Signature size is $308 + 16q$ bytes (where $q \geq 1$ is the signature index), yielding 324 bytes for the first signature, which is $\times 20$ smaller than the smallest SLH-DSA variant.

- **Robust stateless fallback.** When the state is lost, corrupted, or exhausted, SHRINCS falls back to a SPHINCS+-based stateless signature (3912 bytes). This is still smaller than standard SLH-DSA.

- **Static seed backup.** Users can back up their wallet using a standard seed phrase. Upon recovery, the wallet operates in stateless-only mode, preserving funds while accepting larger signatures.

- It provides with two parameters set depending on what matters for verifier:
  - SHRINCS-B (Bitcoin-optimized): Minimizes signature size at the cost of more verification hashes.
  - SHRINCS-L (Liquid-optimized): Minimizes verification cost at the cost of larger signatures.

1.2 Limitations and Practical Considerations

Users and developers should be aware of the following limitations inherent to stateful signature schemes:

State Management Across Devices. The signing state (specifically, the counter q indicating which one-time keys have been already used) cannot be safely shared across multiple devices without coordination. Two devices signing with the same q value would reuse a one-time signature, which leads to the key is compromised. Some mitigation strategies can be:

- Dedicate disjoint ranges of leaf indices to different devices.

- Use a single signing device with secure state storage (other devices are stateless backups).

- Accept stateless-only operation for multi-device wallets.

State Corruption and Power Failure If a signing operation is interrupted (e.g., due to power failure) after producing a signature but before persisting the updated state, the signer may not know whether the one-time key was used. Subsequent signing attempts risk catastrophic key reuse. Approaches for mitigation:

- Incrementing the state counter before the signature. If signing fails, a leaf is wasted but security is preserved.
- State corruption detection: Maintain checksums or redundant state copies to detect corruption. If corruption is detected, fall back to stateless mode.[1]

State Growth with Multiple Addresses Each SHRINCS-based address requires its own signing state. As users generate new addresses (as recommended for privacy), they must maintain state for each address that may be used for future spending. This state must be preserved for as long as the address holds funds. Mitigation:

- Wallet backups must include current state for all active addresses, not just the seed.
- Addresses can be marked as "stateless-only" after backup, accepting larger signatures for those addresses.

Variable Stateful Signature Sizes SHRINKS signatures presume variable signature size and verification complexity depending on: (1) the number of already produced stateful signatures; (2) the signing mode. In decentralized system we need to consider how the cost of the signature verification is calculated to prevent spam attacks.

- That's basically the reason this specification introduced two modes for SHRINCS. SHRINCS-B is designed for accounting systems, where the user pays for each transaction byte, so the signature size matters. SHRINCS-L suits better for systems which calculate the transaction cost based on how many computation units it consumes (like Liquid[2]).

2 Notation

Throughout this specification:

`||` denotes byte string concatenation

`[a:b]` denotes a substring from byte index `a` to `(b-1)`

`[0:a]` denotes first `a` bytes (truncation)

`←$` uniform random sampling

`n` is the security parameter in bytes

`H` denotes a cryptographic hash function

`Tw` denotes a tweakable hash function

`PRF` denotes a pseudorandom function

$\perp$ used for failure returns

[1] Here is a nice guide how to manage a state and backup for post-quantum constructions: https://datatracker.ietf.org/doc/draft-ietf-pquip-hbs-state/

[2] **Oleks:** Make a reference how WU are calculated for liquid and why signatures with begger sizes can result to cheaper transactions

3 Overview

Mike: I wonder if we should include guidlince for state management here. Maybe this: https://datatracker.ietf.org/doc/draft-ietf-pquip-hbs-state/02/ can be useful. But maybe it is not for this document. Maybe it is better to update our HB paper with these discussions.

SHRINCS is designed in the way it can operate in two modes: **stateful** and **stateless**. Stateful mode is represented by the instance of Unbalanced XMSS (UXMSS) with a root public key `PK.sf`. An optimized variant of SPHINCS+ is used to support the stateless operating mode (which is represented by the public key `PK.sl`). The root public key is a hashed concatenation of (`PK.sf`, `PK.sl`), which is being converted to the Bitcoin address.

During normal operation with intact state, the user generates signatures according to efficient stateful path. Each leaf of the UXMSS is represented by WOTS+C one time signature instance. UXMSS is right-skewed, which makes the first stateful signature (`q=1`) the cheapest from the verification and size perspective (324 bytes) and adds the constant size (16 bytes) and one additional hashing operation for each next stateful signature. A right-skewed UXMSS is a tree construction in which every internal node has a one-time signature leaf as its left child and a UXMSS subtree as its right child, yielding a recursively right-branching authentication tree (as shown on the Figure 1).

If the state is corrupted (or the limit for stateful operations is met), the signature can continue operating with stateless mode. Stateless mode utilizes an optimized SPHINCS+ variant with WOTS+C signatures in regular XMSS trees on each layer in the hypertree and FORS+C for few-time signature scheme. The general construction of SHRINCS signature is presented on the Figure 1.

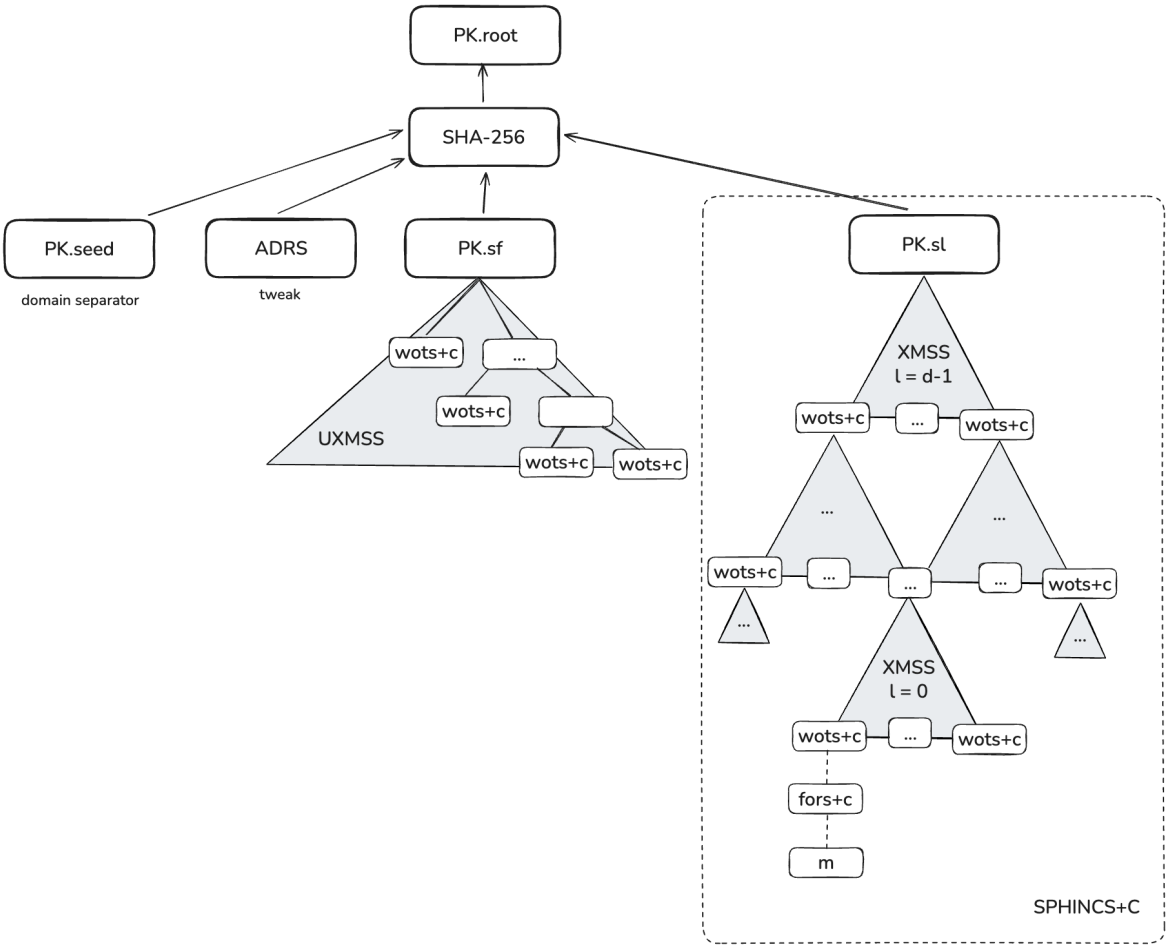

Figure 1: SHRINCS architecture

3.1 Key Pair Derivation and Size

SHRINCS keys are derived from a single random seed (**Oleks:** to add notes about BIP-39 usage and hd wallets if needed, maybe in appendix):

```
seed ←$ {0,1}^(3n)
```

Seed allows to derive a private key `SK ← (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)`:

```
SK.seed ← seed[0:n]  // Seed for key derivation
SK.prf ← seed[n:2n]  // Secret needed for PRF_msg
PK.seed ← seed[2n:3n] // Domain separator for different SHRINCS key pairs
PK.sf ← compute_stateful_root(PK.seed, ADRS, SK.seed) // Public stateful key, see Section X
PK.sl ← compute_stateless_root(PK.seed, ADRS, SK.seed) // Public stateless key, see Section X
PK.root ← H(PK.seed, ADRS, PK.sf, PK.sl) // Root public key
```

where `ADRS` is a tweak also known as a context address (see Section 5.5).

The size of the secret key is `6n` bytes. A public key consists of two values `PK ← (PK.seed, PK.root)` and has a size of `2n` bytes.

Mike: bits, bot bytes

3.2 Signature Size and Verification Cost

3.2.1 Stateful

SHRINCS-B:

- Size: 256 + 16 + 4 + q * 16 = 276 + 16q bytes

  Mike: I think we need to put formulas like w*l + r + ctr + q*hash. Randomness should be 32. Maybe we should add indices that point to which leaf we are using. It is redundant, but i think it is better for the implementation.

- Verification cost: 2042 hashes

  Mike: Verification hashes should depend on q. Also we need to define q in Notation

SHRINCS-L:

Mike: I am not sure what is the difference between SHRINCS-B and SHRINCS-L. Are these two modes of opertaions? With different parameter sets. If so - I think we should add some more explanation to the text.

- Size: 1024 + 16 + 4 + q * 16 = 1044 + 16q bytes

  Mike: Same as above, I would add the formula for computing.

- Verification cost: 54 hashes

  Mike: Same as above - verification depends on q.

3.2.2 Stateless

Oleks: in progress

SHRINCS-B:

- Size:
- Verification cost:

SHRINCS-L:

- Size:
- Verification cost:

4 Parameters

This specification provides with the list of parameters for different SHRINCS instances, depending on verifier requirements:

1. SHRINCS-B: size matters (instance preferable for Bitcoin).
2. SHRINCS-L: verification complexity matters (the number of hashes to verify the signature).

Mike: We can also explore the option of pushing Target Sum value higher (espessilay for smaller SHRINCS-L parameters to make verification even cheaper)

Parameter	SHRINCS-B	SHRINCS-L	Description
`H`	`SHA-256`	`SHA-256`	Hash function
`OTS`	`WOTS+C`	`WOTS+C`	One-time signature
`FTS`	`FORS+C`	`FORS+C`	Few-time signature
`n`	16	16	Security parameter (bytes)
`w`	256	4	Winternitz parameter
`l`	16	64	The number of WOTS+C chains
`Swn`	2040	140	Target sum for WOTS+C
`hsf`	226	418	Maximum stateful tree height
`hsl`	40	40	Maximum stateless hypertree height
`d`	5	5	Stateless hypertree layers
`a`	22	22	FORS tree height
`k`	6	6	Number of FORS+C trees
`c`	4	4	Counter size for WOTS+C grinding (bytes)

5 Underlying Functions and Addresses

5.1 Hash Function

SHRINCS uses SHA-256 truncated to `n` bytes

Mike: Sometimes n appears to be bytes, but sometimes it is used for bits. Should unify.

for all internal hash operations.

```
H(m) = SHA-256(m)[0:n]
```

5.2 Tweakable Hash Function

To prevent multi-target attacks and ensure domain separation between the various components, we employ the tweakable hash function construction defined in FIPS 205 [**fips-205**]

Mike: It was introduced in https://sphincs.org/data/sphincs+-paper.pdf

:

```
Tw(PK.seed, ADRS, m) = H(PK.seed || ADRS || m)
```

This usage of tweaked hashing is critical. It ensures that a hash computed at one position in the Merkle tree cannot be substituted for a hash at another position, effectively mitigating generic tree-grafting attacks.

Oleks: `PK.seed = 16 bytes, ADRS = 32 bytes, m = 16-32 bytes.` SHA-2 operates with 64-byte blocks. So we can add a "0"-padding, like `Tw(PK.seed, ADRS, m) = H(PK.seed || 0⃗[0,3n] || ADRS || m)` to have the first 64 bytes constant. It will reduce the hashing complexity to 1 block for each tweak function.

5.3 Pseudorandom Functions

We use PRF for deterministic key derivation:

```
PRF(PK.seed, ADRS, SK.seed) = Tw(PK.seed, ADRS, SK.seed)
```

Additionally we use a pseudorandom function to generate the randomizer `R` for the randomized hashing of the message to be signed.

```
PRF_msg(SK.prf, PK.seed, opt_rand, m) = HMAC-SHA-256(SK.prf, PK.seed || opt_rand || m),
```

Mike: We will need to plug a counter into PRF msg to be able to grind for FORS+C

where `opt_rand` is an optional randomness (`n` bytes), can be `PK.seed` for deterministic signing.

5.4 Message Hash Function

To generate the digest of the message to be signed we use the following function:

```
H_msg(ADRS, R, PK.seed, PK.root, m) = H(ADRS || R || PK.seed || PK.root || m)
```

5.5 Tweak Structure and Domain Separation

Tweaks are structured as follows to ensure uniqueness across all scheme components:

```
ADRS = layer || tree_address || type || words
```

where:

`layer`: (4 bytes) layer in hypertree (`0` = bottom, `d-1` = max)

`tree_address`: (12 bytes) tree address within layer

`type`: (4 bytes) address type

`words`: (12 bytes) type-dependent 12 bytes

A total `ADRS` size is 32 bytes.

5.5.1 Address Types

`0x00`: Stateful WOTS+C chains (`SF_WOTS_CHAIN`)

`0x01`: Stateful tree nodes (`SF_TREE`)

`0x02`: Stateless WOTS+C chains (`SL_WOTS_CHAIN`)

`0x03`: Stateless tree nodes (`SL_TREE`)

`0x04`: FORS (`SL_FORS`)

`0x05`: Message hash (`H_MSG`)

`0x06`: Pseudorandom function (`PRF`)

`0x07`: Root public key (`ROOT`)

Summarizing, `(0x00, 0x01)` are stateful only types, `(0x02, 0x03, 0x04)` are stateless only and `(0x05, 0x06, 0x07)` are shared types.

Mike: I think SLH-DSA has more ADRS types, such as public-key compression address, FORS tree roots compression address, and we also need need two types for grinding WOTS and for grinding FORS

5.5.2 Address Manipulation Functions

```
setLayerAddress(ADRS, layer):
    ADRS[0:4] ← toByte(layer, 4)

setTreeAddress(ADRS, tree_addr):
    ADRS[4:16] ← toByte(tree_addr, 12)

setTypeAndClear(ADRS, type):
    ADRS[16:20] ← toByte(type, 4)
    ADRS[20:32] ← toByte(0, 12)

setKeyPairAddress(ADRS, keypair):
    ADRS[20:24] ← toByte(keypair, 4)

setChainAddress(ADRS, chain):
    ADRS[24:28] ← toByte(chain, 4)

setHashAddress(ADRS, hash):
    ADRS[28:32] ← toByte(hash, 4)

setTreeHeight(ADRS, height):
    ADRS[24:28] ← toByte(height, 4)

setTreeIndex(ADRS, index):
    ADRS[28:32] ← toByte(index, 4)
```

Note that `setTreeHeight` and `setChainAddress` both write to bytes `[24:28]`. These operations are context-dependent so there mustn't be a conflict.

5.5.3 Type-Dependent Words

Type	Meaning	word1 [20:24]	word2 [24:28]	word3 [28:32]
0x00	SF_WOTS_CHAIN	keypair	chain	hash
0x01	SF_TREE	keypair	height	index
0x02	SL_WOTS_CHAIN	keypair	chain	hash
0x03	SL_TREE	keypair	height	index
0x04	SL_FORS	tree_idx	height	index
0x05	H_MSG	0	0	0
0x06	PRF	W: keypair / F: tree_idx	W: chain / F: 0	W: 0 / F: leaf_idx
0x07	ROOT	0	0	0

Table 1: Type-dependent interpretation of ADRS words (bytes [20:32]). W is a WOTS an F is a FORS

6 WOTS+C

The standard Winternitz One-Time Signature (WOTS+) is the building block of most modern hash-based schemes. It works by encoding the message into a series of small integers and using them to iterate hash chains [**wots+**]. A significant portion of a WOTS+ signature size (approx 20%) is dedicated to "Checksum," which is necessary to prevent a forgery attack where an adversary extends the hash chains forward. WOTS+C (compressed WOTS+) eliminates the transmission of this checksum entirely, achieving a smaller signature size.

In WOTS+, the checksum C is calculated from the message digest D.

$$C = \sum_{i=1}^{l} (w - 1 - d_i)$$

This checksum is then signed using additional chains. In WOTS+C[**wots+c**], instead of calculating a checksum for a fixed message digest, the signer modifies the message digest until it possesses a specific, pre-determined checksum property. This is done by appending a counter `ctr` and a random salt `R`

Mike: I think we should separate those, first we use Hmsg to hash the message with R, and they we use that digest and hash it again with the counter until we find a good digest. I can try to investigate if it is possible to combine those, but it looks like two separate hashes is better.

to the message before hashing. The signer increments `ctr` until the resulting hash digest D satisfies a global target condition.

6.1 The Target Condition

WOTS utilize a parameter `w` to define the number of hashchains and correspondingly their length. The message digest length is `n` bytes so `l = 8n/(log2(w))`. The values $d_{i \in [l]}$ are treated as integers in `0...(w-1)`.

Let `Swn` be a protocol parameter (often near the expected value, but may be chosen larger to trade signing/grinding cost for verification cost). If the hash output is uniform, the expected value of a single digit is `d_i = (w-1)/2`. For `l` digits, the expected `Swn` is:

$$S_{wn} = \sum_{i=0}^{l-1} d_i$$

By enforcing this condition during signing (including grinding), the verifier can simply assume the checksum is `Swn` without receiving it. If the reconstructed digest does not sum to `Swn`, the signature is invalid.

Additionally we can manipulate a target condition to decrease the cost of signature verification. The signature generation and verification operations together consist of `St = (w-1)*l` hashing operations

Mike: Plus grinding, Plus public key compression but we can also put that cost to the tree generation part. Moreover, during signing in the XMSS we need to compute all the chains in the WOTS anyways, since we need to get to public key to compute the leaf of the Merkle tree.

. So we can increase `Swn` (grinding complexity) and receive the reduced verification complexity of `(St - Swn)` hashes. Concrete parameters are presented in Section 4.

6.2 Base-w Representation

WOTS represents a message digest in base `w` so that each digit is an integer in `[0...w-1]`. These digits directly determine the number of hashes to apply to each WOTS chain element during signing and verification. The `base_w()` function converts a byte string into an array of `out_len` base-w digits (this requires `w` to be a power of two.)

Mike: It does not have to be a power of two, but it is more convinient, if it is. So yes, in this document we can assume for it to be a power of 2, but I would change the wording since it is not a strict requirement

.

```
Algorithm: base_w(m, w, out_len)
    m: byte string
    w: Winternitz parameter
    out_len: output length
Output:
    b_w: array of out_len integers in [0, w-1]

1. in_idx ← 0
```

```
2. bits ← 0
3. total ← 0
4. b_w ← []
5. for i from 0 to (out_len - 1):
    a. if bits == 0:
        total ← m[in_idx]
        in_idx ← in_idx + 1
        bits ← 8
    b. bits ← bits - log2(w)
    c. b_w[i] ← (total >> bits) AND (w - 1)
6. return b_w
```

We provide optimized versions for `w = 256` and `w = 4` proposed by different SHRINCS instances. For `w=256`:

```
Algorithm: base_w256(m, out_len)
Input:
    m: byte string
    out_len: output length
Output:
    b_w: array of out_len integers in [0, 255]

1. b_w ← []
2. for i from 0 to (out_len - 1):
    a. b_w[i] ← m[i]
3. return b_w
```

For `w=4`:

```
Algorithm: base_w4(m, out_len)
Input:
    m: byte string
    out_len: output length
Output:
    b_w: array of out_len integers in [0,3]

1. b_w ← []
2. for j from 0 to (out_len/4 - 1):
    a. x ← m[j]
    b. b_w[4*j + 0] ← (x >> 6) AND 3
    c. b_w[4*j + 1] ← (x >> 4) AND 3
    d. b_w[4*j + 2] ← (x >> 2) AND 3
    e. b_w[4*j + 3] ← x AND 3
3. return b_w
```

6.3 Chain Function

WOTS derives signature components and public key material by iteratively applying the tweakable hash function along a Winternitz chain. The function `chain()` takes an n-byte input value `m` and applies `Tw(PK.seed, ADRS, ·)` exactly `steps` times, beginning at chain position `start`. For each iteration, the address field hash inside `ADRS` is set to the current chain position `i` (from `start` to `start+steps-1`) to ensure domain separation between different chain steps. The output is the resulting n-byte value after advancing the chain by `steps`, which is used both for producing signature chain values (advancing from 0 to `msg[i]`) and for reconstructing end-of-chain values during verification (advancing from `msg[i]` to `w-1`).

```
Algorithm: chain(m, start, steps, PK.seed, ADRS)
Input:
```

```
    m: input
    start: starting index
    steps: number of iterations
    PK.seed: public seed
    ADRS: tweak
Output:
    n-byte chain output

1. tmp ← m
2. for i from start to (start + steps - 1):
    a. setHashAddress(ADRS, i)
    b. tmp ← Tw(PK.seed, ADRS, tmp)
3. return tmp
```

6.4 Key Generation

To create a WOTS public key for a given `keypair` index, we derive one secret starting value per chain using `PRF`, then advance each value to the end of its chain. The resulting array of end-of-chain values is compressed to a fixed-size WOTS public key for that `keypair`.

```
Algorithm: wots_PKgen(SK.seed, PK.seed, ADRS, keypair)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
    keypair: keypair index
Output:
    pk: compressed WOTS+C public key

1. tmp ← []
2. for i from 0 to l - 1:
    a. setTypeAndClear(ADRS, PRF)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
    d. sk_i ← PRF(PK.seed, ADRS, SK.seed)
    e. setTypeAndClear(ADRS, SF_WOTS_CHAIN)
    f. setKeyPairAddress(ADRS, keypair)
    g. setChainAddress(ADRS, i)
    h. tmp[i] ← chain(sk_i, 0, w - 1, PK.seed, ADRS)
3. setTypeAndClear(ADRS, SF_TREE)
4. setKeyPairAddress(ADRS, keypair)
5. pk ← Tw(PK.seed, ADRS, tmp[0] || ... || tmp[l-1])
6. return pk
```

Mike: Was the addrs for WOTS compression defined before? The computation of pk is never explained I think

6.5 Messsage Digest Grinding

Mike: Here we need to split message hashing into two parts. First is message digest computation with Randomness. Next hashing of the digest with the counter. I think this is better for security proofs and presents a better modular design. Because in stateless branch you will also hash for WOTS+C where you will not use any randomness, just the previous hash and the counter. Important remark for future: message hashing for SPHINCS+C can differ in output length from the hashing for XMSS. But if we dont put the counter in neither cases we will have the same inouts.

WOTS+C enforces the checksum/target condition by searching over the counter `ctr`. The value `R` (a randomness/nonce input to message hashing) may be fixed (deterministic) and need not be resampled during grinding. The function `wots_grind()` increments `ctr` and recomputes the message digest until the derived base-w digits satisfy the target condition. It outputs the found `ctr` and the corresponding digest (and `R` is carried in the signature only if verification requires it to recompute the same digest).

Oleks: To consider if we need to mention counter in Section 5.4

```
Algorithm: wots_grind(m, R, PK.seed, PK.root, ADRS)
Input:
    m: message
    R: randomness
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    msg: array of l integers
    ctr: final counter value used

1. setTypeAndClear(ADRS, H_MSG)
2. for ctr from 0 to 2^32 - 1:
    a. digest ← H_msg(ADRS, R || ctr, PK.seed, PK.root, m)
    b. msg ← base_w(digest, l)
    c. sum ← 0
    d. for i from 0 to (l - 1):
        sum ← sum + msg[i]
    e. if sum == Swn:
        return (msg, ctr)
3. abort with error  // R must be resampled
```

Oleks: to add expected grinding complexity for different approaches

Additionally we need to define a func which allows to calculate the digest without grinding (by providing the `ctr` value directly).

```
Algorithm: wots_digest(m, R, ctr, PK.seed, PK.root, ADRS)
Input:
    m: message
    R: randomness
    ctr: counter
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    msg: array of l integers
    valid: boolean indicating if sum == Swn

1. setTypeAndClear(ADRS, H_MSG)
2. digest ← H_msg(ADRS, R || ctr, PK.seed, PK.root, m)
3. msg ← base_w(digest, l)
4. sum ← 0
5. for i from 0 to l - 1:
    sum ← sum + msg[i]
6. valid ← (sum == Swn)
7. return (msg, valid)
```

6.6 Signature Generation

To sign, WOTS+C instance computes the message digits and then reveals, for each chain, the chain value after exactly `msg[i]` steps (starting from the secret-derived chain `start`). These revealed values

form the WOTS signature vector. The signature also carries the randomness `R` and grinding counter `ctr` needed for the verifier to reproduce the same message digits.

Mike: Also split the message hashing here.

```
Algorithm: wots_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, keypair)
Input:
    m: message
    SK.seed: secret seed
    SK.prf: PRF key for randomness
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
    keypair: keypair index
Output:
    sig: WOTS+C signature (R || ctr || chain_values)

1. opt_rand ← random(n) or PK.seed  // for randomized or deterministic signing
2. R ← PRF_msg(SK.prf, PK.seed, opt_rand, m)
3. (msg, ctr) ← wots_grind(m, R, PK.seed, PK.root, ADRS)
4. sig ← R || toByte(ctr, c)
5. for i from 0 to (l - 1):
    a. setTypeAndClear(ADRS, PRF)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
    d. sk_i ← PRF(PK.seed, ADRS, SK.seed)
    e. setTypeAndClear(ADRS, SF_WOTS_CHAIN)
    f. setKeyPairAddress(ADRS, keypair)
    g. setChainAddress(ADRS, i)
    h. sig ← sig || chain(sk_i, 0, msg[i], PK.seed, ADRS)
6. return sig
```

6.7 Signature Verification (Public Key Recovery)

Verification reconstructs the WOTS+C public key implied by the signature. For each chain element, the verifier continues hashing forward from the signature's provided intermediate value for `w-1-msg[i]` steps to reach the end-of-chain value. Compressing all end-of-chain values yields the recovered WOTS+C public key, which can then be checked against the expected value in the surrounding scheme.

Mike: Message hash

```
Algorithm: wots_pkFromSig(sig, m, PK.seed, PK.root, ADRS, keypair, mode)
Input:
    sig: signature
    m: signed message
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
    keypair: keypair index
Output:
    pk: recovered public key or $\bot$ if invalid

1. R ← sig[0:R_SIZE]                 // maybe it makes sense to define sig_parse op
2. ctr ← sig[R_SIZE:R_SIZE + c]
3. chains_start ← R_SIZE + c
4. (msg, valid) ← wots_digest(m, R, ctr, PK.seed, PK.root, ADRS)
5. if not valid:
```

```
        return $\bot$
    6. tmp ← []
    7. for i from 0 to (l - 1):
        a. setTypeAndClear(ADRS, SF_WOTS_CHAIN)
        b. setKeyPairAddress(ADRS, keypair)
        c. setChainAddress(ADRS, i)
        d. sig_i ← sig[chains_start + i * n : chains_start + (i+1)*n]
        e. tmp[i] ← chain(sig_i, msg[i], w - 1 - msg[i], PK.seed, ADRS)
    8. setTypeAndClear(ADRS, SF_TREE)
    9. setKeyPairAddress(ADRS, keypair)
    10. pk ← Tw(PK.seed, ADRS, tmp[0] || ... || tmp[l-1])
    11. return pk
```

6.8 Contribution to the Signature Size and Verification Complexity

6.9 Implementation notes

In this section we have defined basic WOTS+C operations for stateful tree instance. But actually the same one time signature is used in stateless instance too. So we recommend to have a `mode` parameter that defines the `ADRS` with `SF_WOTS_CHAIN / SL_WOTS_CHAIN` and `SF_TREE / SL_TREE` correspondingly.

7 XMSS

SHRINCS employs two distinct Merkle tree constructions: a standard balanced XMSS tree for the stateless hypertree, and an Unbalanced XMSS (UXMSS) tree optimized for the stateful component. We will specify both.

The stateless signature mode uses a SPHINCS+-style hypertree consisting of `d` layers, where each layer contains standard balanced XMSS trees. Each XMSS tree authenticates `pow(2,h_prime)` WOTS+C key pairs, where `h_prime = hsl/d` is the height of trees within each layer.

7.1 XMSS Tree Structure

An XMSS tree of height `h'` is a complete binary Merkle tree where:

- Leaves: The `pow(2,h_prime)`leaves are WOTS+C public keys, computed via `wots_PKgen`.
- Internal nodes: Each internal node is the tweakable hash of its two children.
- Root: The tree root serves as either the layer's contribution to the hypertree or (for the top layer) as `PK.sl`.

Layer indexing within the stateless hypertree is the following:

- Layer 0: Bottom layer (is used to sign the FORS+C public keys)
- Layer d-1: Top layer (root is `PK.sl`)

7.2 XMSS TreeHash

The `xmss_treehash` function computes subtree roots within an XMSS tree. It takes a target height and a starting leaf index, then recursively builds the subtree by computing leaves (WOTS+C public keys) at height 0 and hashing pairs of children to form internal nodes. This function is the core building block used both for computing the full tree root (by calling it with `target_height = h_prime` and `start_idx = 0`) and for generating individual nodes needed in authentication paths.

Mike: I see that you require here that the start idx must be zero, but don't we need a general xmss treehash alg that takes a specific leaf used to sign the message and does the same as your xmss treehash. See Algorithm 9 xmss node in SLH-DSA spec. If we decide to keep this as a function I would propose name change for the variable: leftmost leaf index (smth like this)

```
Algorithm: xmss_treehash(SK.seed, PK.seed, ADRS, target_height, start_idx)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: address (with layer and tree address already set)
    target_height: height of output node (0 = leaf level)
    start_idx: leftmost leaf index in subtree
Output:
    node: hash value (n bytes)

1. if target_height == 0:
    a. pk ← wots_PKgen(SK.seed, PK.seed, ADRS, start_idx)
    b. return pk
2. left ← xmss_treehash(SK.seed, PK.seed, ADRS, target_height - 1, start_idx)
3. right ← xmss_treehash(SK.seed, PK.seed, ADRS, target_height - 1, start_idx + 2^(target_height
4. setTypeAndClear(ADRS, SL_TREE)
5. setKeyPairAddress(ADRS, start_idx >> target_height)
6. setTreeHeight(ADRS, target_height)
7. setTreeIndex(ADRS, start_idx >> target_height)
8. return Tw(PK.seed, ADRS, left || right)
```

7.3 XMSS Root Computation

The `xmss_root` function computes the root of an entire XMSS tree by invoking `xmss_treehash` with the full tree height and starting from leaf index 0. The resulting root hash represents the public key for this XMSS tree instance, which either becomes `PK.sl` (for the top hypertree layer) or is signed by the layer above (for lower layers).

```
Algorithm: xmss_root(SK.seed, PK.seed, ADRS, h_prime)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: address (layer and tree address set)
    h_prime: tree height
Output:
    root: tree root (n bytes)

1. return xmss_treehash(SK.seed, PK.seed, ADRS, h_prime, 0)
```

7.4 XMSS Authentication Path Generation

For a leaf at index `idx`, the authentication path consists of the sibling nodes at each level from the leaf up to (but not including) the root. At height `h`, the sibling is determined by flipping the h-th bit of idx using XOR. The authentication path contains `h_prime` nodes, each of size `n` bytes, allowing a verifier to reconstruct the path from the leaf to the root. The sibling at each level is computed by calling `xmss_treehash` on the subtree rooted at that sibling. For a leaf at index `idx`, the authentication path consists of sibling nodes at each level from the leaf to just below the root.

```
Algorithm: xmss_auth_path(SK.seed, PK.seed, ADRS, h_prime, idx)
Input:
    SK.seed, PK.seed: seeds
    ADRS: address (layer and tree address set)
    h_prime: tree height
    idx: leaf index being authenticated
Output:
    auth: authentication path (h_prime * n bytes)
```

```
1. auth ← []
2. for h from 0 to h_prime - 1:
    a. sibling_idx ← idx XOR (1 << h)
    b. sibling_start ← (sibling_idx >> h) << h
    c. auth ← auth || xmss_treehash(SK.seed, PK.seed, ADRS, h, sibling_start)
3. return auth
```

Mike: 2.b feels unnecessary if we update the xmss treehash function.

7.5 XMSS Public Key Recovery from Signature

Given a WOTS+C signature and authentication path, this algorithm recovers the XMSS tree root. First, it recovers the WOTS+C public key (the leaf value) from the WOTS+C signature. Then, starting from the leaf, it iteratively combines the current node with the corresponding authentication path node to compute the parent, continuing until reaching the root. At each height `h`, the `h`-th bit of `idx` determines whether the current node is the left child (bit = 0) or right child (bit = 1), which dictates the ordering when hashing the pair.

Mike: And here we write a different implementation istead of reusing xmss treehash algorithm. Probably mainly because the treehash recuired the starting index to be 0.

```
Algorithm: xmss_pkFromSig(wots_sig, auth, m, PK.seed, PK.root, ADRS, h_prime, idx)
Input:
    wots_sig: WOTS+C signature
    auth: authentication path (h_prime * n bytes)
    m: signed message
    PK.seed: public seed
    PK.root: root public key
    ADRS: address (layer and tree address set)
    h_prime: tree height
    idx: leaf index
Output:
    root: computed tree root (n bytes), or $\bot$ if invalid

1. pk ← wots_pkFromSig(wots_sig, m, PK.seed, PK.root, ADRS, idx)
2. if pk == $\bot$:
    return $\bot$
3. node ← pk
4. for h from 0 to h_prime - 1:
    a. setTypeAndClear(ADRS, SL_TREE)
    b. setKeyPairAddress(ADRS, idx >> 1)
    c. setTreeHeight(ADRS, h + 1)
    d. setTreeIndex(ADRS, idx >> 1)
    e. auth_node ← auth[h * n : (h + 1) * n]
    f. if (idx AND 1) == 0:
        node ← Tw(PK.seed, ADRS, node || auth_node)
    g. else:
        node ← Tw(PK.seed, ADRS, auth_node || node)
    h. idx ← idx >> 1
5. return node
```

7.6 XMSS Signature Generation (Stateless Layer)

To sign a message within the stateless hypertree, the XMSS signature generation produces a WOTS+C signature on the message followed by the authentication path for the corresponding leaf. The message being signed is typically either a FORS+C public key (at layer 0) or the root of a lower-layer XMSS tree

(at layers 1 through d-1). The leaf index `idx` determines which WOTS+C key pair is used; this index is determined from the message digest during stateless signing.

```
Algorithm: xmss_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, h_prime, idx)
Input:
    m: message to sign (lower layer root or FORS public key)
    SK.seed: secret seed
    SK.prf: PRF key
    PK.seed: public seed
    PK.root: root public key
    ADRS: address
    h_prime: tree height
    idx: leaf index to use
Output:
    sig: XMSS signature (WOTS+C signature || authentication path)

1. wots_sig ← wots_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, idx)
2. auth ← xmss_auth_path(SK.seed, PK.seed, ADRS, h_prime, idx)
3. return wots_sig || auth
```

8 UXMSS. The Stateful Tree

Standard XMSS uses a balanced binary tree, which minimizes the maximum path length but imposes a logarithmic cost on every signature. For `N` signatures, every signature has size proportional to `log_2(N)`. SHRINCS utilizes an Unbalanced Merkle Tree (specifically a "Right-Skewed" tree) to optimize for the typical Bitcoin use case: a wallet that performs very few transactions in its lifecycle (we are targeting `N<=10`).

In this topology the `q`-th signature requires a path of length `q`. This linear growth is superior to logarithmic growth for small `q`. For `q=1`, the path is just 16 bytes. Even for `q=10`, the path is $10 \times 16 = 160$ bytes, which is extremely small compared to the fixed $\approx$ 2.5KB of a balanced tree or the $\approx$ 8KB of SLH-DSA.

8.1 UXMSS Tree Structure

Oleks: I hope it's not confusing we have layers numeration different for stateless and statefull instance

Mike: I meean, it is possible to understand, but I think it is better to unify the numbering.

The UXMSS tree is constructed recursively as follows:

- Level 0 is the root public key `PK.sf`.
- Level `i` (internal):
  - Left child: WOTS+C public key for signature `q=i`
  - Right child: Root of subtree for signatures `q>i`
- Level `hsf` (bottom)
  - Left child: WOTS+C public key for signature `q=hsf`
  - Right child: WOTS+C public key for signature `q=hsf+1`

8.2 UXMSS TreeHash

The `uxmss_treehash` function computes subtree roots within the right-skewed UXMSS tree. Unlike balanced XMSS where treehash recursively computes both subtrees symmetrically, UXMSS treehash follows the right-skewed structure: the left child is always a WOTS+C public key (a leaf), while the

right child is either another UXMSS subtree (for internal levels) or a second WOTS+C public key (at the bottom level). The level parameter indicates depth from the root, with level 0 being the root and level `hsf-1` being the bottom internal node.

Mike: Should we have a unified API for these functions with similar logic: xmss treehash(SK.seed, PK.seed, ADRS, target height, start idx) Is level the target hash that is output? If Lvl is 1 what do we output: WOTSpk or rightsubtree.

```
Algorithm: uxmss_treehash(SK.seed, PK.seed, ADRS, level)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: address (layer=0, tree_address=0)
    level: level in tree (0 = root, hsf-1 = bottom internal node)
Output:
    node: hash value (n bytes)

1. left ← wots_PKgen(SK.seed, PK.seed, ADRS, level + 1)

2. if level == hsf - 1:
        right ← wots_PKgen(SK.seed, PK.seed, ADRS, hsf + 1)
   else:
        right ← uxmss_treehash(SK.seed, PK.seed, ADRS, level + 1)
3. setTypeAndClear(ADRS, SF_TREE)
4. setTreeHeight(ADRS, hsf - level)
5. setTreeIndex(ADRS, 0)
6. return Tw(PK.seed, ADRS, left || right)
```

8.3 UXMSS Root Computation

The `uxmss_root` function computes `PK.sf`, the root of the stateful UXMSS tree. It initializes the address structure with layer 0 and tree address 0 (since there is only one stateful tree), then invokes `uxmss_treehash` starting from level 0. The resulting root, combined with `PK.sl` from the stateless tree, forms the basis for `PK.root`.

```
Algorithm: uxmss_root(SK.seed, PK.seed, ADRS)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: address
Output:
    PK.sf: stateful public key (n bytes)

1. setLayerAddress(ADRS, 0)
2. setTreeAddress(ADRS, 0)
3. return uxmss_treehash(SK.seed, PK.seed, ADRS, 0)
```

8.4 UXMSS Authentication Path Generation

For signature number `1 <= q <= hsf + 1`, the authentication path contains exactly `q` sibling nodes needed to reconstruct `PK.sf` from the WOTS+C public key at position `q`. The structure of the path differs based on whether `q <= hsf` or `q = hsf + 1`. For `q <= hsf`, the first sibling is either a subtree root (if `q < hsf`) or the rightmost WOTS+C key (if `q = hsf`), followed by q-1 WOTS+C public keys ascending toward the root. For `q = hsf + 1` (the rightmost leaf), all hsf siblings are WOTS+C public keys. This asymmetric structure reflects the right-skewed nature of UXMSS.

Mike: I think we still can use idx variable, and just use it as a pointer to the leaf, counted from left to right.

```
    Algorithm: uxmss_auth_path(SK.seed, PK.seed, ADRS, q)
    Input:
        SK.seed, PK.seed: seeds
        ADRS: address
        q: signature number (1 to hsf + 1)
    Output:
        auth: authentication path (q * n bytes)

    1. auth ← []
    2. setLayerAddress(ADRS, 0)
    3. setTreeAddress(ADRS, 0)
    4. if q <= hsf:
            if q == hsf:
                auth ← auth || wots_PKgen(SK.seed, PK.seed, ADRS, hsf + 1)
            else:
                auth ← auth || uxmss_treehash(SK.seed, PK.seed, ADRS, q)
            for i from 1 to q - 1:
                auth ← auth || wots_PKgen(SK.seed, PK.seed, ADRS, q - i)
    5. else:
            for i from 0 to hsf - 1:
                auth ← auth || wots_PKgen(SK.seed, PK.seed, ADRS, hsf - i)
6. return auth
```

8.5 UXMSS Public Key Recovery from Signature

Given a WOTS+C signature and authentication path, this algorithm reconstructs `PK.sf`. First, it recovers the WOTS+C public key from the signature. Then, starting from this leaf, it iteratively combines the current node with authentication path nodes to compute parent nodes until reaching the root. The combination order depends on the position q: for `q <= hsf`, the first combination has the recovered key on the left and the subtree/sibling on the right, while subsequent combinations have the sibling (a WOTS+C key from a lower q value) on the left. For `q = hsf + 1`, all siblings are on the left since this is the rightmost path through the tree.

Mike: The checks at 7.c can be just moved outside the loop. First sibling is always left, then it is always right.

```
    Algorithm: uxmss_pkFromSig(wots_sig, auth, m, PK.seed, PK.root, ADRS, q)
    Input:
        wots_sig: WOTS+C signature
        auth: authentication path (q * n bytes)
        m: signed message
        PK.seed: public seed
        PK.root: root public key
        ADRS: address
        q: signature number (1 to hsf + 1)
    Output:
        root: computed PK.sf (n bytes), or $\bot$ if invalid

    1. setLayerAddress(ADRS, 0)
    2. setTreeAddress(ADRS, 0)
    3. pk ← wots_pkFromSig(wots_sig, m, PK.seed, PK.root, ADRS, q)
    4. if pk == $\bot$:
       return $\bot$
    5. node ← pk
    6. setTypeAndClear(ADRS, SF_TREE)
    7. if q <= hsf:
        for i from 0 to q - 1:
```

```
        a. setTreeHeight(ADRS, hsf - (q - 1 - i))
        b. setTreeIndex(ADRS, 0)
        c. if i == 0:
               node ← Tw(PK.seed, ADRS, node || auth[i])
           else:
               node ← Tw(PK.seed, ADRS, auth[i] || node)
8. else:
   for i from 0 to hsf - 1:
        a. setTreeHeight(ADRS, i + 1)
        b. setTreeIndex(ADRS, 0)
        c. // auth[i] is always left (WOTS), node is right
           node ← Tw(PK.seed, ADRS, auth[i] || node)
9. return node
```

8.6 UXMSS Signature Generation

To sign a message using the stateful path, UXMSS signature generation produces a WOTS+C signature followed by the authentication path for position q. The signature number q must be tracked as state and incremented after each use to ensure one-time signature security. The resulting signature size is `|wots_sig| + q * n` bytes, which grows linearly with q. For the first signature (q=1), this yields the minimal 324-byte signature that makes SHRINCS attractive for Bitcoin applications.

```
Algorithm: uxmss_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, q)
Input:
    m: message to sign
    SK.seed: secret seed
    SK.prf: PRF key
    PK.seed: public seed
    PK.root: root public key
    ADRS: address
    q: signature number to use (1 to hsf + 1)
Output:
    sig: UXMSS signature

1. setLayerAddress(ADRS, 0)
2. setTreeAddress(ADRS, 0)
3. wots_sig ← wots_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, q)
4. auth ← uxmss_auth_path(SK.seed, PK.seed, ADRS, q)
5. return wots_sig || auth
```

9 FORS+C

FORS (Forest of Random Subsets) is a few-time signature scheme used at the bottom of the stateless hypertree. FORS allows to sign a limited number of messages, when the security degrades with each use. Standard FORS signs a message by revealing `k` secret values, one from each of `k` binary trees of height `a`. The message digest determines which leaf to reveal from each tree. FORS+C applies the same checksum grinding optimization as WOTS+C to reduce signature size.

9.1 FORS Structure

FORS consists of `k` binary trees, each of height `a`:

- Each tree has 2^a leaves
- Each leaf is derived from `SK.seed` using PRF
- The message digest is split into `k` chunks of `a` bits each

- Each chunk selects one leaf index from the corresponding tree

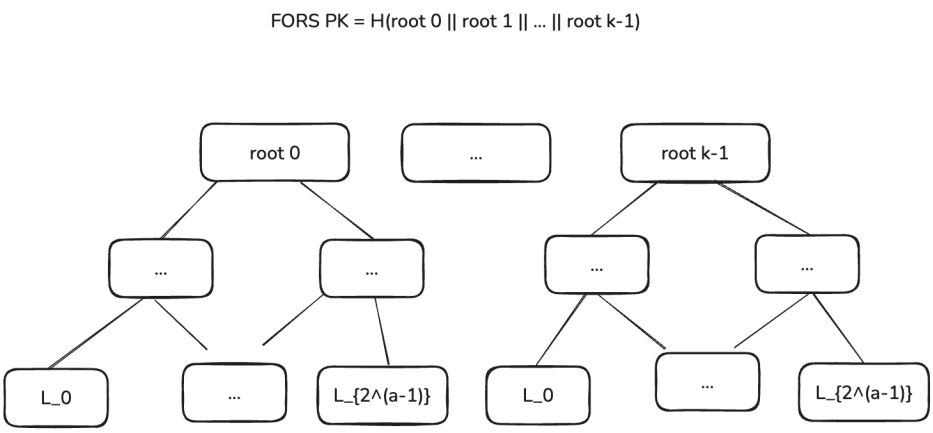

Figure 2: FORS architecture

Mike: We put the counter into PRFmsg for the stateless option, and regenerate the randomness each time. PRFmsg(inputcs, counter)= R. Hmsg(inputs, R)= digest, check if good, otherwise restart from PRFmsg with a new counter.

9.2 Grinding Condition

```
digest = H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
condition: digest[last a bits] == 0
```

Cost is expected to be $\approx 2^a$ iterations.

9.3 Message to Indices

The `fors_msg_to_indices` function converts a message digest into k leaf indices, one for each FORS tree.

Mike: The mesage digest also contains the index of the FORS instance that is used. We also need to make sure that the output size of the function is sufficient for this encodings.

Each index is extracted as a consecutive a-bit chunk from the digest, selecting which leaf to reveal from the corresponding tree. The indices are integers in the range `[0, pow(2,a) - 1]`.

```
Algorithm: fors_msg_to_indices(digest)
Input:
    digest: message digest
Output:
    indices: array of k integers in [0, 2^a - 1]

1. indices ← []
2. for i from 0 to k - 1:
    a. indices[i] ← extract_bits(digest, i * a, a)
3. return indices
```

9.4 Grinding

The `fors_grind` function searches for a counter value where the last tree's index equals 0. This eliminates the need to send the authentication path for tree k-1, since when index 0 is selected, the verifier can

simply receive the tree root directly. The function iterates through counter values until finding one that produces a digest whose last a bits are zero.

Mike: This can be variable. we can require one, two, or more indices to be 0. Also need to adapt the message hashing, and indices extraction.

```
Algorithm: fors_grind(m, R, PK.seed, PK.root, ADRS)
Input:
    m: message to sign
    R: randomness
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    indices: array of k integers
    ctr: counter value

1. setTypeAndClear(ADRS, H_MSG)
2. for ctr from 0 to 2^32 - 1:
    a. digest ← H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
    b. indices ← fors_msg_to_indices(digest)
    c. if indices[k-1] == 0:
        return (indices, ctr)
3. abort with error
```

9.5 Secret Key Generation

The `fors_SKgen` function derives a secret leaf value for a specific tree and leaf index. Each leaf's secret is deterministically generated using PRF with the tree index and leaf index encoded in the address structure. This ensures all `pow(2,a)` leaves across all k trees can be regenerated from `SK.seed` without storing them.

```
Algorithm: fors_SKgen(SK.seed, PK.seed, ADRS, tree_idx, leaf_idx)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
    tree_idx: tree index (0 to k-1)
    leaf_idx: leaf index within tree
Output:
    sk: secret value (n bytes)

1. setTypeAndClear(ADRS, PRF)
2. setKeyPairAddress(ADRS, tree_idx)
3. setTreeIndex(ADRS, leaf_idx)
4. sk ← PRF(PK.seed, ADRS, SK.seed)
5. return sk
```

9.6 TreeHash

The `fors_treehash` function recursively computes FORS subtree roots. At height 0 (leaf level), it generates the secret key and hashes it to produce the leaf node. At higher levels, it recursively computes left and right children, then hashes their concatenation. This function is used both for computing full tree roots and for generating individual nodes needed in authentication paths.

Mike: This is similar to xmss treehash. So maybe we can reuse some code, and the same comments on the start idx apply here. And for algorithms below, I think if we modularise things in the right way we can just reuse the xmss functions everywhere here.

```
Algorithm: fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, target_height, start_idx)
Input:
    SK.seed, PK.seed: seeds
    ADRS: tweak
    tree_idx: tree index (0 to k-1)
    target_height: height of output node
    start_idx: leftmost leaf index in subtree
Output:
    node: hash value (n bytes)

1. if target_height == 0:
    a. sk ← fors_SKgen(SK.seed, PK.seed, ADRS, tree_idx, start_idx)
    b. setTypeAndClear(ADRS, SL_FORS)
    c. setKeyPairAddress(ADRS, tree_idx)
    d. setTreeHeight(ADRS, 0)
    e. setTreeIndex(ADRS, start_idx)
    f. return Tw(PK.seed, ADRS, sk)
2. left ← fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, target_height-1, start_idx)
3. right ← fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, target_height-1, start_idx +
            2^(target_height-1))
4. setTypeAndClear(ADRS, SL_FORS)
5. setKeyPairAddress(ADRS, tree_idx)
6. setTreeHeight(ADRS, target_height)
7. setTreeIndex(ADRS, start_idx >> target_height)
8. return Tw(PK.seed, ADRS, left || right)
```

9.7 FORS Authentication path

The `fors_auth_path` function generates the a sibling nodes needed to authenticate a revealed leaf up to its tree root. At each height h, the sibling is the node that, when combined with the current path node, produces the parent. The sibling index is computed by XORing the leaf index with `pow(2,h)`, which flips the h-th bit to identify the sibling's subtree.

```
Algorithm: fors_auth_path(SK.seed, PK.seed, ADRS, tree_idx, leaf_idx)
Input:
    SK.seed, PK.seed: seeds
    ADRS: tweak
    tree_idx: tree index (0 to k-1)
    leaf_idx: leaf index being revealed
Output:
    auth: authentication path (a * n bytes)

1. auth ← []
2. for h from 0 to a - 1:
    a. sibling_start ← (leaf_idx XOR 2^h) AND (0xFFFFFFFF << h)
    b. auth ← auth || fors_treehash(SK.seed, PK.seed, ADRS, tree_idx, h, sibling_start)
3. return auth
```

9.8 Signature generation

The `fors_sign` function produces a FORS+C signature by revealing k-1 leaves with their full authentication paths, plus a special case for tree k-1. Due to grinding, the last tree's index is guaranteed to be 0, so instead of revealing leaf 0 and its authentication path, the signature includes only the tree root.

Mike: We don't even need the tree root here. We just completly exclude it from the signature. the FORS+C pk = H(root0, ..., rootk-2). Line 6 can be droped.

This saves `(n + a * n) - n = a * n` bytes compared to standard FORS. The signature format is: `R || ctr || (sk_0 || auth_0) || ... || (sk_k-2 || auth_k-2) || root_k-1`.

```
Algorithm: fors_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS)
Input:
    m: message
    SK.seed: secret seed
    SK.prf: PRF key
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    sig: FORS+C signature

1. opt_rand ← random(n) or PK.seed
2. R ← PRF_msg(SK.prf, PK.seed, opt_rand, m)
3. (indices, ctr) ← fors_grind(m, R, PK.seed, PK.root, ADRS)
4. sig ← R || toByte(ctr, c)
5. for i from 0 to k - 2:
    a. sk_i ← fors_SKgen(SK.seed, PK.seed, ADRS, i, indices[i])
    b. auth_i ← fors_auth_path(SK.seed, PK.seed, ADRS, i, indices[i])
    c. sig ← sig || sk_i || auth_i
6. return sig
```

9.9 Public Key Recovery

Mike: here we can exclude the last tree root, same as above. The algorithm presented under line 9 should be a subfunction fors treehash?

The `fors_pkFromSig` function recovers the FORS public key from a signature. For each of the first k-1 trees, it hashes the revealed secret key to obtain the leaf, then iteratively combines with authentication path nodes to compute the tree root. For tree k-1, the root is directly extracted from the signature (since grinding ensures index 0 was selected). Finally, all k tree roots are hashed together to produce the FORS public key. The function returns $\perp$ if the grinding condition is not satisfied (`indices[k-1] != 0`).

```
Algorithm: fors_pkFromSig(sig, m, PK.seed, PK.root, ADRS)
Input:
    sig: FORS+C signature
    m: message
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    pk: FORS public key (n bytes), or $\bot$ if invalid

1. R ← sig[0 : n]
2. ctr ← sig[n : n + c]
3. setTypeAndClear(ADRS, H_MSG)
4. digest ← H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
5. indices ← fors_msg_to_indices(digest)
6. if indices[k-1] != 0:
    return $\bot$
7. roots ← []
8. offset ← n + c
9. for i from 0 to k - 2:
    a. sk_i ← sig[offset : offset + n]
```

```
    b. auth_i ← sig[offset + n : offset + n + a * n]
    c. offset ← offset + n + a * n
    d. setTypeAndClear(ADRS, SL_FORS)
    e. setKeyPairAddress(ADRS, i)
    f. setTreeHeight(ADRS, 0)
    g. setTreeIndex(ADRS, indices[i])
    j. node ← Tw(PK.seed, ADRS, sk_i)
    k. idx ← indices[i]
    l. for h from 0 to a - 1:
        setTreeHeight(ADRS, h + 1)
        setTreeIndex(ADRS, idx >> 1)
        if (idx AND 1) == 0:
            node ← Tw(PK.seed, ADRS, node || auth_i[h * n : (h+1) * n])
        else:
            node ← Tw(PK.seed, ADRS, auth_i[h * n : (h+1) * n] || node)
        idx ← idx >> 1
    m. roots[i] ← node
10. setTypeAndClear(ADRS, SL_FORS)
11. setTreeHeight(ADRS, a)
12. setTreeIndex(ADRS, 0)
13. pk ← Tw(PK.seed, ADRS, roots[0] || ... || roots[k-2])
14. return pk
```

10 SHRINCS

This section specifies the complete SHRINCS signature scheme by composing the building blocks defined in previous sections.

10.1 Key Generation

The `SHRINCS.KeyGen` function generates a fresh key pair and initializes the signing state. It samples a 3n-byte random seed, partitions it into `SK.seed` (for key derivation), `SK.prf` (for message randomization), and `PK.seed` (for domain separation). It then computes `PK.sf` (the stateful UXMSS root) and `PK.sl` (the stateless hypertree root), combining them into `PK.root` via a tweakable hash. The initial state sets `q=0` with `valid=true`, indicating the stateful path is available starting from signature number 1.

```
Algorithm: SHRINCS.KeyGen()
Output:
    SK: secret key
    PK: public key
    state: initial signing state

1. seed ←$ {0,1}^(3n)
2. SK.seed ← seed[0:n]
3. SK.prf ← seed[n:2n]
4. PK.seed ← seed[2n:3n]
5. ADRS ← new_ADRS()
6. PK.sf ← uxmss_root(SK.seed, PK.seed, ADRS)
7. setLayerAddress(ADRS, d - 1)
8. setTreeAddress(ADRS, 0)
9. PK.sl ← xmss_root(SK.seed, PK.seed, ADRS, h_prime)
10. setTypeAndClear(ADRS, ROOT)
11. PK.root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
12. SK ← (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)
13. PK ← (PK.seed, PK.root)
14. state ← { q: 0, valid: true }
```

```
15. return (SK, PK, state)
```

10.2 Recovery from Seed

The `SHRINCS.Restore` function recovers a key pair from a backed-up seed. This enables the standard Bitcoin workflow of restoring a wallet from a seed phrase. However, since the signing state (which q values have been used) cannot be recovered from the seed alone, the stateful path is disabled by setting `valid=false`. The recovered wallet can only use stateless signatures, which are larger but do not require state tracking. This is the key tradeoff that enables static backups while maintaining security.

```
Algorithm: SHRINCS.Restore(seed)
Input:
    seed: master seed (3n bytes)
Output:
    SK: secret key
    PK: public key
    state: invalid state (stateless only)

1. SK.seed ← seed[0:n]
2. SK.prf ← seed[n:2n]
3. PK.seed ← seed[2n:3n]
4. ADRS ← new_ADRS()
5. PK.sf ← uxmss_root(SK.seed, PK.seed, ADRS)
6. setLayerAddress(ADRS, d - 1)
7. setTreeAddress(ADRS, 0)
8. PK.sl ← xmss_root(SK.seed, PK.seed, ADRS, h')
9. setTypeAndClear(ADRS, ROOT)
10. PK.root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
11. SK ← (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)
12. PK ← (PK.seed, PK.root)
13. state ← { q: $\bot$, valid: false }
14. return (SK, PK, state)
```

10.3 Statefull signing

The `SHRINCS.SignStateful` function signs a message using the efficient stateful (UXMSS) path. It first checks that state is valid and that signature slots remain (`q <= hsf + 1`). It then increments q, generates a UXMSS signature at position q, and appends PK.sl to enable unified verification. The updated state is returned alongside the signature. This produces minimal signatures (324 bytes for `q=1`) that grow linearly with q. If state is invalid or exhausted, the function returns $\perp$ and the caller should use stateless signing instead.

Mike: We need to first update the state then run signing. Otherwise if the program is iterupted we get a signature, but not the updated state.

```
Algorithm: SHRINCS.SignStateful(m, SK, state)
Input:
    m: message to sign
    SK: secret key (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)
    state: current signing state { q, valid }
Output:
    sig: stateful signature, or $\bot$ if state is corrupted
    state': updated state

1. if not state.valid:
    return ($\bot$, state)
2. q ← state.q + 1
```

```
3. if q > hsf + 1:
    return ($\bot$, state)
4. ADRS ← new_ADRS()
5. uxmss_sig ← uxmss_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, q)
6. sig ← PK.sl || uxmss_sig
7. state' ← { q: q, valid: true }
8. return (sig, state')
```

10.4 Stateless signing

The `SHRINCS.SignStateless` function signs a message using the stateless (SPHINCS+C) path. This is used when state is lost, corrupted, or exhausted. It generates a randomized message digest, extracts tree and leaf indices for each hypertree layer, then constructs a signature bottom-up: first a FORS+C signature on the message, then XMSS signatures on each layer's root ascending through the hypertree. The signature includes `PK.sf` to enable unified verification. Stateless signatures are larger but can be generated without any state management.

Mike: It looks like we first compute the digest to determine the index, and then we compute the digest again to determine the revealed leaves in the FORS. I think we should not split that. We should use the same digest computation to determine both the instance of FORS and its leaves. Check here: https://github.com/eyalr0/sphincsplusc.

```
Algorithm: SHRINCS.SignStateless(m, SK)
Input:
    m: message to sign
    SK: secret key (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)
Output:
    sig: stateless signature

1. ADRS ← new_ADRS()
2. opt_rand ← random(n) or PK.seed
3. R ← PRF_msg(SK.prf, PK.seed, opt_rand, m)
4. setTypeAndClear(ADRS, H_MSG)
5. digest ← H_msg(ADRS, R, 0, PK.seed, PK.root, m)
6. (tree_idx, leaf_idx) ← parse_idx(digest, hsl, d)
7. setLayerAddress(ADRS, 0)
8. setTreeAddress(ADRS, tree_idx[0])
9. fors_sig ← fors_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS)
10. fors_pk ← fors_pkFromSig(fors_sig, m, PK.seed, PK.root, ADRS)
11. ht_sig ← []
12. msg ← fors_pk
13. for layer from 0 to d - 1:
    a. setLayerAddress(ADRS, layer)
    b. setTreeAddress(ADRS, tree_idx[layer])
    c. xmss_sig ← xmss_sign(msg, SK.seed, SK.prf, PK.seed, PK.root, ADRS, h', leaf_idx[layer])
    d. ht_sig ← ht_sig || xmss_sig
    e. if layer < d - 1:
        msg ← xmss_root(SK.seed, PK.seed, ADRS, h')
14. sig ← PK.sf || fors_sig || ht_sig
15. return sig
```

10.5 Index Parsing

The `parse_idx` function extracts tree and leaf indices for each hypertree layer from the message digest. The digest is interpreted as a large integer, then partitioned into d pairs of (`leaf_idx, tree_idx`). Each leaf index selects which WOTS+C key pair to use within a tree, while each tree index identifies which

tree within the layer. The indices are extracted from least significant (layer 0) to most significant (layer `d-1`), with h' bits per leaf index and decreasing bits for tree indices at higher layers.

Mike: This is a bit confusin, that we extract the indices of the OTS/FTS schemes used, but not the indices of the revealed leaves in FORS. With the Hashing changes, it needs a double-check that everything is computed correctly and the same data is not used for two purposes.

```
Algorithm: parse_idx(digest, hsl, d)
Input:
    digest: message digest
    hsl: total hypertree height
    d: number of layers
Output:
    tree_idx: array of d tree indices
    leaf_idx: array of d leaf indices

1. h_prime ← hsl / d
2. tree_idx ← []
3. leaf_idx ← []
4. idx ← bytes_to_int(digest[0 : (hsl + 7) / 8])
5. for layer from 0 to d - 1:
    a. leaf_idx[layer] ← idx AND (2^h_prime - 1)
    b. idx ← idx >> h_prime
    c. tree_idx[layer] ← idx AND (2^(hsl - (layer + 1) * h_prime) - 1)
6. return (tree_idx, leaf_idx)
```

10.6 Statefull verification

Mike: Needs updates that correspond to updates in message hashing

The `SHRINCS.VerifyStateful` function verifies a stateful signature. It extracts `PK.sl` from the signature tail, parses the UXMSS signature, and infers q from the authentication path length. It then recovers PK.sf using `uxmss_pkFromSig` and verifies that `H(PK.sf || PK.sl)` equals the expected `PK.root`. The signature is valid if and only if the WOTS+C verification succeeds and the reconstructed root matches.

```
Algorithm: SHRINCS.VerifyStateful(m, sig, PK)
Input:
    m: message
    sig: stateful signature
    PK: public key (PK.seed, PK.root)
Output:
    valid: boolean

1. ADRS ← new_ADRS()
2. sig_len ← len(sig)
3. PK.sl ← sig[0 : n]
4. uxmss_sig ← sig[n : sig_len]
5. wots_sig_size ← R_SIZE + c + l * n
6. auth_len ← len(uxmss_sig) - wots_sig_size
7. q ← auth_len / n
8. if q < 1 or q > hsf + 1:
    return false
9. wots_sig ← uxmss_sig[0 : wots_sig_size]
10. auth ← uxmss_sig[wots_sig_size : wots_sig_size + q * n]
11. PK.sf ← uxmss_pkFromSig(wots_sig, auth, m, PK.seed, PK.root, ADRS, q)
12. if PK.sf == $\bot$:
    return false
```

```
13. setLayerAddress(ADRS, d - 1)
14. setTypeAndClear(ADRS, ROOT)
15. expected_root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
16. return (expected_root == PK.root)
```

10.7 Stateless verification

The `SHRINCS.VerifyStateless` function verifies a stateless signature. It extracts `PK.sf` from the signature head, parses the FORS+C signature, recomputes the message digest and indices, then verifies bottom-up through the hypertree. Starting with the FORS+C public key recovery, it iteratively verifies each XMSS layer by recovering the tree root and using it as the message for the next layer. The final recovered root becomes `PK.sl`, which is combined with `PK.sf` to verify against `PK.root`.

Mike: Here the Hmsg compuations do not match the signing prcedure.

```
Algorithm: SHRINCS.VerifyStateless(m, sig, PK)
Input:
    m: message
    sig: stateless signature
    PK: public key (PK.seed, PK.root)
Output:
    valid: boolean

1. ADRS ← new_ADRS()
2. PK.sf ← sig[0 : n]
3. offset ← n
4. fors_sig_size ← R_SIZE + c + (k - 1) * (n + a * n)
5. fors_sig ← sig[offset : offset + fors_sig_size]
6. offset ← offset + fors_sig_size
7. R ← fors_sig[0 : R_SIZE]
8. ctr ← fors_sig[R_SIZE : R_SIZE + c]
9. setTypeAndClear(ADRS, H_MSG)
10. digest ← H_msg(ADRS, R, ctr, PK.seed, PK.root, m)
11. (tree_idx, leaf_idx) ← parse_idx(digest, hsl, d)
12. setLayerAddress(ADRS, 0)
13. setTreeAddress(ADRS, tree_idx[0])
14. fors_pk ← fors_pkFromSig(fors_sig, m, PK.seed, PK.root, ADRS)
15. if fors_pk == $\bot$:
    return false
16. msg ← fors_pk
17. xmss_sig_size ← R_SIZE + c + l * n + h' * n
18. for layer from 0 to d - 1:
    a. xmss_sig ← sig[offset : offset + xmss_sig_size]
    b. offset ← offset + xmss_sig_size
    c. wots_sig ← xmss_sig[0 : R_SIZE + c + l * n]
    d. auth ← xmss_sig[R_SIZE + c + l * n : ]
    e. setLayerAddress(ADRS, layer)
    f. setTreeAddress(ADRS, tree_idx[layer])
    g. msg ← xmss_pkFromSig(wots_sig, auth, msg, PK.seed, PK.root, ADRS, h', leaf_idx[layer])
    h. if msg == $\bot$:
        return false
19. PK.sl ← msg
20. setTypeAndClear(ADRS, ROOT)
21. expected_root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
22. return (expected_root == PK.root)
```

10.8 Unified Verification

The `SHRINCS.Verify` function provides a single entry point for verifying either stateful or stateless signatures. It distinguishes between the two types based on signature length: stateful signatures are bounded by `max_sf_size` (the maximum stateful signature size for `q = hsf + 1`), while stateless signatures exceed this threshold. This allows verifiers to accept both signature types without requiring explicit type indicators, simplifying the verification interface.

```
Algorithm: SHRINCS.Verify(m, sig, PK)
Input:
    m: message
    sig: signature (stateful or stateless)
    PK: public key (PK.seed, PK.root)
Output:
    valid: boolean

if len(sig) <= max_sf_size:
    return SHRINCS.VerifyStateful(m, sig, PK)
else:
    return SHRINCS.VerifyStateless(m, sig, PK)
```

11 Data structures

11.1 Secret Key Format

11.2 Public Key Format

11.3 Stateful Signature Format

```
SHRINCS-B:
    PK.sl : 16 bytes    //public key of stateless instance
    c : 4 bytes. //
    chain_values : l * 16 = 16 * 16 = 256 bytes
    merkle_path : q * 16 bytes
    R : 16 bytes
```

Total size: 292 + q * 16 bytes

```
SHRINCS-L:
    PK.sl : 16 bytes    //public key of stateless instance
    c : 4 bytes. //
    chain_values : l * 16 = 64 * 16 = 1024 bytes
    merkle_path : q * 16 bytes
    R : 16 bytes
```

Total size: 1060 + q * 16 bytes

11.4 Stateless Signature Format